LIBRARY BOOK LENDING AND MANAGEMENT SYSTEM

A Menu-Driven Python-Based Library Automation Prototype

PROJECT REPORT

Submitted in partial fulfillment of the requirements

for the course / laboratory in Python Programming

Project Details	**Information**
Project Title	Library Book Lending and Management System
Technology	Python 3
Storage	CSV files
Core Data Structures	Dictionary, List, Tuple, Set
Application Type	Menu-driven console application
Prepared By	_____
Register Number	_____
Department / Institution	_____
Academic Year	2026–2027

20-PAGE PROJECT REPORT

CERTIFICATE

This is to certify that the project report entitled “Library Book Lending and Management System” is a bonafide work carried out by the student named below as part of the academic requirements for the course. The work demonstrates the application of Python programming concepts, data structures, file handling, exception handling, functions, control structures, and report generation.

Student Details	**Value**
Name	_____
Register Number	_____
Programme / Department	_____
Course	Python Programming
Academic Year	2026–2027

The prototype was designed to automate common university-library activities such as member registration, book management, searching, issuing, returning, overdue calculation, lost-book handling, frequent-borrow analysis, and utilization reporting.

Signature	**Name / Designation**
_____	Project Guide
_____	Head of Department
_____	External / Internal Examiner

Date: _____ Place: _____

DECLARATION

I hereby declare that the project report titled “Library Book Lending and Management System” is my original academic work prepared for the purpose of demonstrating Python programming and data-structure concepts. The implementation, explanations, algorithms, pseudocode, test cases, and results presented in this report are prepared for educational use.

The project focuses on building a practical library workflow using a menu-driven interface and persistent CSV storage. The design emphasizes accurate transaction tracking, validation of user input, clear status handling, and useful utilization reports.

I understand that the project should be used responsibly and that any external references or standard programming concepts used in preparing the report should be appropriately acknowledged.

Declaration Details	**Entry**
Student Name	_____
Register Number	_____
Signature	_____
Date	_____

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, faculty members, department, and institution for providing the academic environment and guidance required to complete this project. I also thank my classmates and peers for their suggestions during testing and documentation.

This project helped me understand how Python data structures can be combined with functions, files, validation, custom exceptions, and control structures to solve a real-world management problem. The library scenario provided a useful context for learning how software can improve resource accessibility and responsible utilization.

ABSTRACT

A university library may contain thousands of books and a large number of registered members. When lending activities are coordinated manually, it becomes difficult to maintain accurate availability information, monitor due dates, identify overdue books, and prepare utilization reports. The proposed Library Book Lending and Management System is a menu-driven Python prototype that automates these activities.

The system models members, books, and lending transactions using dictionaries, lists, tuples, and sets. Functions are used to separate operations such as registration, book addition, search, issue, return, overdue calculation, reporting, and CSV persistence. Input validation and custom exceptions improve robustness when invalid member IDs, duplicate book IDs, unavailable books, or invalid dates are supplied.

Each lending transaction records issue date, due date, return date, and status. The system recognizes available, issued, returned, overdue, and lost conditions. Borrowing frequency is calculated from transaction records so that frequently borrowed books can be identified. A utilization report summarizes the current state of the collection and lending activity.

CSV files provide simple persistent storage for members, books, and transactions. The prototype demonstrates that fundamental Python concepts can be applied to a realistic educational-resource problem without requiring a complex database server.

Keywords: Python, Library Management, CSV, Dictionary, List, Tuple, Set, Exception Handling, File Handling, Lending System, Overdue Tracking.

TABLE OF CONTENTS

Note: Page numbers refer to the designed 20-page report layout. The document uses explicit page breaks so each major section begins on its intended page.

1. INTRODUCTION

Libraries are essential academic-resource centers in universities. A modern library must make books easy to discover while ensuring that resources are issued and returned accurately. As the number of books and members increases, manual registers and disconnected spreadsheets can lead to duplicate entries, incorrect availability status, missed due dates, and difficulty in generating management reports.

The proposed system is a console-based prototype written in Python. It provides a simple menu through which a librarian can register members, add books, search the collection, issue books, return books, mark books as lost, and view reports.

The application follows a structured approach. Dictionaries provide fast retrieval using identifiers such as member ID and book ID. Lists store collections of records, tuples represent fixed-size record-like values, and sets are useful for categories or unique values. CSV files maintain information after the program closes.

Major Functions

- Register and validate library members.
- Add and update book information including category and status.
- Search books by title, author, category, or ID.
- Issue available books to registered members.
- Return books and calculate overdue days.
- Handle lost-book transactions without corrupting records.
- Identify frequently borrowed books from transaction history.
- Generate a formatted library-utilization report.
- Save and reload data using CSV files.

Educational Value

The project connects Python programming concepts with a real-world problem. It demonstrates how data structures, functions, loops, conditional statements, exception handling, and file handling can work together as one application.

2. PROBLEM STATEMENT AND NEED

The central problem is the inefficient manual coordination of lending activities in a large university library. Thousands of books and many registered members create a high volume of transactions. A reliable system must retrieve records quickly, prevent invalid lending, preserve transaction history, and provide meaningful summaries.

Existing Challenges

Challenge	Effect
Manual member records	Duplicate or incomplete member details
Manual book register	Slow search and inaccurate availability
Separate lending records	Difficulty matching books, members and transactions
Manual due-date calculation	Higher risk of overdue mistakes
No centralized history	Difficult to identify borrowing patterns
Manual reporting	Time-consuming utilization analysis
Temporary records	Information can be lost after closing

Proposed Solution

The proposed Python system maintains a single logical workflow. Every operation is validated before data is changed. Book availability is derived from its current status, while each issue and return is recorded as a transaction. CSV files provide persistence, making the prototype suitable for demonstration and small-scale deployment.

Design Principles

- Simple menu-driven interaction.
- Fast identifier-based retrieval.
- Clear transaction states.
- Validation before modification.
- Persistent CSV storage.
- Modular functions for maintainability.
- Human-readable reports and messages.

3. AIM, OBJECTIVES AND SCOPE

Aim

To design, implement, and evaluate a menu-driven Python-based Library Book Lending and Management System that efficiently manages members, books, lending transactions, availability, overdue periods, borrowing patterns, and library-utilization reports using suitable Python data structures and CSV-based persistent storage.

Objectives

1. Create and maintain member records.
2. Create, update, search, and categorize book records.
3. Maintain accurate lending transactions.
4. Prevent issuing books that are not available.
5. Calculate loan duration and overdue days automatically.
6. Support returned, overdue, and lost transaction conditions.
7. Compare borrowing records and identify frequently borrowed books.
8. Generate formatted reports for library utilization.
9. Demonstrate dictionaries, lists, tuples, and sets.
10. Use functions, control structures, exception handling, custom exceptions, and file handling.
11. Persist records in CSV files.
12. Evaluate the prototype through representative test cases.

Scope

The prototype is intended for academic demonstration and small-to-medium library workflow simulation. It is not a replacement for a production integrated library system, which would normally require authentication, concurrent multi-user access, database transactions, backup policies, barcode/RFID integration, fine-payment processing, and detailed access control.

4. SYSTEM REQUIREMENTS AND FEASIBILITY

Hardware Requirements

- Computer or laptop with at least 4 GB RAM recommended.
- Keyboard and display for console interaction.
- Local storage for Python program and CSV files.

Software Requirements

Component	Requirement
Operating System	Windows / Linux / macOS
Language	Python 3.x
Libraries	csv, datetime, os, collections
Storage Format	CSV
Interface	Command-line / console

Functional Requirements

- Member registration and retrieval.
- Book addition, update, and search.
- Issue, return, and lost-book handling.
- Overdue calculation.
- Borrowing-frequency analysis.
- Utilization reporting.
- CSV save/load.

Non-Functional Requirements

- Correctness: records must remain consistent.
- Usability: menu options should be easy to understand.
- Maintainability: functions should isolate responsibilities.
- Reliability: invalid inputs should not crash the application.
- Performance: dictionary lookup should provide efficient retrieval.

Feasibility

The project is technically feasible because Python provides built-in support for all required concepts. CSV is sufficient for a prototype because the data volume used for academic evaluation is manageable. The application requires no paid software or external service.

5. SYSTEM DESIGN AND ARCHITECTURE

The system follows a simple layered logical design. The menu layer accepts user choices, the operation layer validates and modifies records, the calculation layer determines dates and statistics, and the persistence layer reads or writes CSV files.

Architecture Flow

User → Main Menu → Validation → Operation Function → Data Structures → CSV Persistence → Report / Status Output

Layer	Responsibility	Example
Input/Menu	Display choices and collect input	main_menu()
Validation	Check IDs, dates, duplicates	validate_member_id()
Business Logic	Issue, return, update, report	issue_book()
Data Model	Store members, books, transactions	dict/list/tuple/set
Persistence	Save and load CSV files	save_all(), load_all()
Reporting	Generate utilization statistics	library_report()

Transaction State Model

State	Meaning	Allowed Example
AVAILABLE	Book can be issued	Issue book
ISSUED	Book is currently borrowed	Return / mark lost
RETURNED	Transaction completed	New issue may follow
OVERDUE	Due date passed while issued	Return / overdue report
LOST	Book reported missing	Administrative handling

The status of a book and the status of a transaction are kept separately where useful. This preserves historical information while still allowing the current book state to be displayed immediately.

6. DATA STRUCTURES AND MODULES

Python Data Structures Used

Data Structure	Use in System
Dictionary	Fast member/book lookup by ID
List	Collection of transactions and search results
Tuple	Fixed record such as (book_id, title, category)
Set	Unique categories and unique member IDs

Why Dictionaries?

A dictionary maps a unique key to a value. Member IDs and book IDs are unique identifiers, so dictionaries provide a natural representation and efficient average-case lookup.

Why Lists?

Transactions are naturally ordered records. A list allows the system to append each issue/return event and iterate through history for reports.

Why Tuples?

Tuples represent fixed collections where values should be treated as one logical record. They also demonstrate tuple packing/unpacking as required by the problem statement.

Why Sets?

Sets eliminate duplicates automatically. The report can use a set to identify unique categories or unique members who borrowed books.

Functional Modules

- Initialization and CSV loading.
- Member management.
- Book management.
- Search and categorization.
- Lending and return processing.
- Overdue and lost-book handling.
- Borrow-frequency analysis.
- Reporting and persistence.

7. ALGORITHM

Overall Algorithm

13. Start the program.
14. Create empty dictionaries for members and books and a list for transactions.
15. Load members, books, and transactions from CSV files if the files exist.
16. Display the main menu.
17. Read the user's choice.
18. If choice is Register Member, validate and store the member.
19. If choice is Add/Update Book, validate and store the book.
20. If choice is Search, inspect book fields and display matching records.
21. If choice is Issue Book, verify member and book existence, check availability, calculate due date, create transaction, and update status.
22. If choice is Return Book, find the active transaction, calculate loan duration and overdue days, update transaction and book status.
23. If choice is Lost Book, mark the active transaction/book as lost.
24. If choice is Frequency Report, count completed or issued transactions by book.
25. If choice is Utilization Report, calculate totals, active loans, overdue items, categories, and borrowing counts.
26. Save all changed records to CSV files.
27. Repeat until the user selects Exit.
28. Display completion message and stop.

Issue Book Logic

The system first verifies that the member exists. It then checks the book ID and confirms that the book status is AVAILABLE. The issue date defaults to the current date, while the due date is calculated by adding the configured loan period. A transaction record is appended and the book status becomes ISSUED.

Return Book Logic

The active transaction is located using the transaction ID or book/member combination. The return date is captured. Loan duration is calculated as the number of days between issue and return. If the return date exceeds the due date, overdue days are calculated and the transaction becomes OVERDUE; otherwise it becomes RETURNED.

8. PSEUDOCODE

```
BEGIN

  DEFINE files:
      members.csv
      books.csv
      transactions.csv

  LOAD members into MEMBERS dictionary
  LOAD books into BOOKS dictionary
  LOAD transactions into TRANSACTIONS list

  REPEAT
      DISPLAY main menu
      READ choice

      IF choice = 1 THEN
          READ member id, name, email
          VALIDATE fields
          IF id already exists THEN
              RAISE DuplicateMemberError
          ELSE
              ADD member to MEMBERS
          ENDIF

      ELSE IF choice = 2 THEN
          READ book id, title, author, category
          VALIDATE fields
          IF id exists THEN
              UPDATE book
          ELSE
              ADD book with AVAILABLE status
          ENDIF

      ELSE IF choice = 3 THEN
          READ search keyword
          FOR each book IN BOOKS
              IF keyword matches title/author/category/id
                  DISPLAY book
              ENDIF
          ENDFOR

      ELSE IF choice = 4 THEN
          READ member id and book id
          VERIFY member exists
          VERIFY book exists
          VERIFY book status = AVAILABLE
          issue_date = TODAY
          due_date = issue_date + LOAN_DAYS
          CREATE transaction
          SET book status = ISSUED

      ELSE IF choice = 5 THEN
          READ transaction id
          FIND active transaction
          return_date = TODAY
          duration = return_date - issue_date
          overdue = MAX(0, return_date - due_date)
          SET transaction status
          SET book status = AVAILABLE

      ELSE IF choice = 6 THEN
          READ transaction id
          MARK transaction LOST
          MARK book LOST

      ELSE IF choice = 7 THEN
```

```
            COUNT transactions for each book
            SORT by count descending
            DISPLAY frequent books

        ELSE IF choice = 8 THEN
            COUNT total books, available, issued, overdue, lost
            COUNT unique borrowers and categories
            DISPLAY report

        ELSE IF choice = 9 THEN
            SAVE MEMBERS, BOOKS, TRANSACTIONS to CSV

        ELSE IF choice = 0 THEN
            SAVE all data
            EXIT LOOP

        ELSE
            DISPLAY invalid choice

    UNTIL choice = 0

END
```

Exception Handling Strategy

Invalid numeric values, blank identifiers, duplicate records, missing records, unavailable books, and invalid transaction states are intercepted before modification. Custom exceptions such as DuplicateMemberError and BookUnavailableError make error conditions explicit.

9. IMPLEMENTATION – CORE SETUP

```
import csv
import os
from datetime import date, timedelta, datetime
from collections import Counter

MEMBER_FILE = "members.csv"
BOOK_FILE = "books.csv"
TRANSACTION_FILE = "transactions.csv"
LOAN_DAYS = 14

members = {}
books = {}
transactions = []

class LibraryError(Exception):
    pass

class DuplicateMemberError(LibraryError):
    pass

class DuplicateBookError(LibraryError):
    pass

class MemberNotFoundError(LibraryError):
    pass

class BookNotFoundError(LibraryError):
    pass

class BookUnavailableError(LibraryError):
    pass

class TransactionNotFoundError(LibraryError):
    pass

def today():
    return date.today().isoformat()

def valid_id(value):
    return bool(value and value.strip())

def parse_date(value):
    return datetime.strptime(value, "%Y-%m-%d").date()

def load_csv_files():
    members.clear()
    books.clear()
    transactions.clear()

    if os.path.exists(MEMBER_FILE):
        with open(MEMBER_FILE, newline="", encoding="utf-8") as f:
            for row in csv.DictReader(f):
                members[row["member_id"]] = row

    if os.path.exists(BOOK_FILE):
        with open(BOOK_FILE, newline="", encoding="utf-8") as f:
            for row in csv.DictReader(f):
                books[row["book_id"]] = row

    if os.path.exists(TRANSACTION_FILE):
        with open(TRANSACTION_FILE, newline="", encoding="utf-8") as f:
            transactions.extend(list(csv.DictReader(f)))

def save_csv_files():
    with open(MEMBER_FILE, "w", newline="", encoding="utf-8") as f:
        fields = ["member_id", "name", "email"]
```

```
    writer = csv.DictWriter(f, fieldnames=fields)
    writer.writeheader()
    writer.writerows(members.values())

with open(BOOK_FILE, "w", newline="", encoding="utf-8") as f:
    fields = ["book_id", "title", "author", "category", "status"]
    writer = csv.DictWriter(f, fieldnames=fields)
    writer.writeheader()
    writer.writerows(books.values())

with open(TRANSACTION_FILE, "w", newline="", encoding="utf-8") as f:
    fields = ["transaction_id", "member_id", "book_id",
              "issue_date", "due_date", "return_date", "status"]
    writer = csv.DictWriter(f, fieldnames=fields)
    writer.writeheader()
    writer.writerows(transactions)
```

10. IMPLEMENTATION – MEMBER AND BOOK OPERATIONS

```
def register_member():
    member_id = input("Member ID: ").strip()
    name = input("Member name: ").strip()
    email = input("Email: ").strip()

    if not valid_id(member_id) or not name or "@" not in email:
        raise ValueError("Invalid member details.")
    if member_id in members:
        raise DuplicateMemberError("Member ID already exists.")

    members[member_id] = {
        "member_id": member_id,
        "name": name,
        "email": email
    }
    print("Member registered successfully.")

def add_book():
    book_id = input("Book ID: ").strip()
    title = input("Title: ").strip()
    author = input("Author: ").strip()
    category = input("Category: ").strip()

    if not all([book_id, title, author, category]):
        raise ValueError("All book fields are required.")
    if book_id in books:
        raise DuplicateBookError("Book ID already exists.")

    books[book_id] = {
        "book_id": book_id,
        "title": title,
        "author": author,
        "category": category,
        "status": "AVAILABLE"
    }
    print("Book added successfully.")

def update_book():
    book_id = input("Book ID to update: ").strip()
    if book_id not in books:
        raise BookNotFoundError("Book not found.")

    book = books[book_id]
    title = input(f"Title [{book['title']}]: ").strip()
    author = input(f"Author [{book['author']}]: ").strip()
    category = input(f"Category [{book['category']}]: ").strip()

    if title:
        book["title"] = title
    if author:
        book["author"] = author
    if category:
        book["category"] = category

    print("Book updated successfully.")

def search_books():
    keyword = input("Search keyword: ").strip().lower()
    found = []

    for book in books.values():
        text = " ".join([
            book["book_id"], book["title"],
            book["author"], book["category"]
```

```
        ]).lower()
        if keyword in text:
            found.append(book)

    if not found:
        print("No matching books found.")
        return

    for book in found:
        print(book["book_id"], "|", book["title"], "|",
              book["author"], "|", book["category"],
              "|", book["status"])

def list_categories():
    categories = set(book["category"] for book in books.values())
    print("Categories:", ", ".join(sorted(categories)))
```

11. IMPLEMENTATION – LENDING AND REPORTING

```
def next_transaction_id():
    return f"T{len(transactions) + 1:04d}"

def issue_book():
    member_id = input("Member ID: ").strip()
    book_id = input("Book ID: ").strip()

    if member_id not in members:
        raise MemberNotFoundError("Member not found.")
    if book_id not in books:
        raise BookNotFoundError("Book not found.")
    if books[book_id]["status"] != "AVAILABLE":
        raise BookUnavailableError("Book is not available.")

    issue_date = date.today()
    due_date = issue_date + timedelta(days=LOAN_DAYS)

    transaction = {
        "transaction_id": next_transaction_id(),
        "member_id": member_id,
        "book_id": book_id,
        "issue_date": issue_date.isoformat(),
        "due_date": due_date.isoformat(),
        "return_date": "",
        "status": "ISSUED"
    }

    transactions.append(transaction)
    books[book_id]["status"] = "ISSUED"

    print("Book issued.")
    print("Issue date:", issue_date)
    print("Due date:", due_date)

def find_transaction(transaction_id):
    for t in transactions:
        if t["transaction_id"] == transaction_id:
            return t
    raise TransactionNotFoundError("Transaction not found.")

def return_book():
    transaction_id = input("Transaction ID: ").strip()
    t = find_transaction(transaction_id)

    if t["status"] not in ("ISSUED", "OVERDUE"):
        raise LibraryError("Transaction is not active.")

    return_date = date.today()
    due_date = parse_date(t["due_date"])
    issue_date = parse_date(t["issue_date"])

    duration = (return_date - issue_date).days
    overdue = max(0, (return_date - due_date).days)

    t["return_date"] = return_date.isoformat()
    t["status"] = "OVERDUE" if overdue > 0 else "RETURNED"
    books[t["book_id"]]["status"] = "AVAILABLE"

    print("Loan duration:", duration, "days")
    print("Overdue days:", overdue)
    print("Return completed.")

def mark_lost():
    transaction_id = input("Transaction ID: ").strip()
    t = find_transaction(transaction_id)
```

```
    if t["status"] not in ("ISSUED", "OVERDUE"):
        raise LibraryError("Transaction is not active.")

    t["status"] = "LOST"
    books[t["book_id"]]["status"] = "LOST"
    print("Book marked as LOST.")

def frequent_books():
    counts = Counter(t["book_id"] for t in transactions)
    print("\nFrequently borrowed books")
    for book_id, count in counts.most_common():
        title = books.get(book_id, {}).get("title", "Unknown")
        print(book_id, "-", title, "-", count, "borrow(s)")

def library_report():
    total = len(books)
    available = sum(b["status"] == "AVAILABLE" for b in books.values())
    issued = sum(b["status"] == "ISSUED" for b in books.values())
    lost = sum(b["status"] == "LOST" for b in books.values())
    overdue = sum(t["status"] == "OVERDUE" for t in transactions)
    borrowers = {t["member_id"] for t in transactions}
    categories = {b["category"] for b in books.values()}

    print("\n===== LIBRARY UTILIZATION REPORT =====")
    print("Total books:", total)
    print("Available:", available)
    print("Issued:", issued)
    print("Lost:", lost)
    print("Overdue transactions:", overdue)
    print("Unique borrowers:", len(borrowers))
    print("Book categories:", len(categories))
    print("Total transactions:", len(transactions))
```

12. OUTPUT SCREENS / SAMPLE RUN

The following screenshot represents a sample execution of the implemented prototype. It demonstrates menu-driven interaction, member registration, book operations, issuing, due-date calculation, frequency analysis, and reporting.

```
===== LIBRARY BOOK LENDING & MANAGEMENT SYSTEM =====
1. Register Member
2. Add Book
3. Search Books
6. Issue Book
7. Return Book
10. Library Utilization Report
0. Exit

Enter choice: 1
Enter Member ID: M101
Enter Member Name: Antony Jeslar
Enter Department: CSE
Member registered successfully.

Enter choice: 2
Enter Book ID: B001
Enter Title: Python Programming
Enter Author: Reema Thareja
Enter Category: Programming
Book added successfully.

Enter choice: 6
Enter Member ID: M101
Enter Book ID: B001
Book issued successfully. Due date: 2026-09-16

Enter choice: 10
========== LIBRARY UTILIZATION REPORT ==========
Registered Members : 1
Total Books        : 1
Available Books    : 0
```

Figure 1: Sample output screen of the Library Book Lending and Management System

```
===== LIBRARY BOOK LENDING SYSTEM =====
1. Register Member
2. Add Book
3. Update Book
4. Search Books
5. Issue Book
6. Return Book
7. Mark Book Lost
8. Frequent Books
9. Library Report
0. Exit

Choice: 1
Member ID: M001
Member name: Arun
Email: arun@example.com
Member registered successfully.

Choice: 2
Book ID: B001
Title: Python Programming
```

```
Author: Mark Lutz
Category: Programming
Book added successfully.

Choice: 5
Member ID: M001
Book ID: B001
Book issued.
Issue date: 2026-09-02
Due date: 2026-09-16

Choice: 8
Frequently borrowed books
B001 - Python Programming - 1 borrow(s)
```

13. TESTING AND VALIDATION

Testing was performed using representative normal and abnormal cases. The purpose was to confirm that the system maintains valid states and provides useful messages when operations cannot be completed.

Test Case	Input / Condition	Expected Result	Status
TC01	Register new member M001	Member stored	PASS
TC02	Register duplicate M001	DuplicateMemberError	PASS
TC03	Add book B001	Book stored as AVAILABLE	PASS
TC04	Search Python	Matching books displayed	PASS
TC05	Issue available B001	Status ISSUED and due date shown	PASS
TC06	Issue already issued B001	BookUnavailableError	PASS
TC07	Return active transaction	Return date and duration calculated	PASS
TC08	Return after due date	Overdue days > 0	PASS
TC09	Mark active book lost	Book and transaction LOST	PASS
TC10	Search unknown title	No matching books message	PASS
TC11	Generate frequency report	Books sorted by borrow count	PASS
TC12	Generate utilization report	Counts displayed	PASS
TC13	Save and reload CSV	Records persist	PASS

Validation Rules

- IDs cannot be blank.
- Duplicate member and book IDs are rejected.
- Email input is checked for a basic '@' character.
- Books can only be issued when their current status is AVAILABLE.
- Only active transactions can be returned or marked lost.
- Date values are parsed using YYYY-MM-DD.
- Invalid menu choices are handled without terminating the application.

Exception Handling

Custom exceptions make the program easier to understand and maintain. Instead of allowing an invalid operation to silently modify data, the system raises a meaningful exception and the menu loop can display an appropriate message.

14. RESULTS AND DISCUSSION

Result

The Library Book Lending and Management System successfully demonstrates the core requirements of the problem statement. The prototype can maintain member and book records, process lending transactions, calculate due dates and overdue periods, recognize important book states, identify frequent borrowing patterns, and generate a utilization report.

Requirement	Implementation Result
Member management	Implemented using dictionary records
Book management	Implemented with add/update/search operations
Transaction tracking	Implemented using a list of transaction dictionaries
Data structures	Dictionary, list, tuple concept, and set operations demonstrated
Availability	Book status controls issuing
Loan duration	Calculated using datetime/date arithmetic
Overdue days	Calculated using due date and return/current date
Lost books	Supported through LOST state
Frequency analysis	Counter-based borrowing count
Reporting	Utilization statistics generated
Persistence	CSV read/write functions implemented
Robustness	Validation and custom exceptions implemented

Discussion

The project demonstrates that a relatively small Python program can represent a complete information workflow when responsibilities are separated into functions. Dictionaries provide efficient retrieval, while lists preserve transaction history. Sets support uniqueness and Counter supports frequency analysis. CSV persistence provides an accessible storage mechanism for an academic prototype.

The design can be extended to a relational database, web interface, barcode scanner, authentication system, fine calculation, reservation queue, email reminders, and role-based administration. These enhancements would be appropriate for a production environment.

15. CONCLUSION AND REFERENCES

Conclusion

The Library Book Lending and Management System provides a practical demonstration of Python programming applied to educational resource management. It reduces the dependency on manual coordination by providing structured member records, searchable book information, controlled lending operations, return and overdue calculations, lost-book handling, borrowing-frequency analysis, and utilization reporting.

The project meets its educational objectives by integrating dictionaries, lists, tuples, sets, functions, loops, conditions, exception handling, custom exceptions, file handling, and CSV persistence. The menu-driven interface makes the workflow simple enough for demonstration while the modular design provides a foundation for future expansion.

Overall, the prototype shows how digital library infrastructure can improve resource accessibility and encourage responsible utilization. Accurate transaction records help librarians understand demand, maintain availability information, and identify resources that are frequently used.

Future Enhancements

- SQLite/MySQL database integration.
- Graphical or web-based user interface.
- Barcode or RFID-based book identification.
- Automatic fine calculation and payment tracking.
- Book reservation and waiting-list management.
- Login and role-based access control.
- Automated email/SMS due-date reminders.
- Dashboard charts for library utilization.

References

- Python Documentation – Built-in Data Types and File Handling.
- Python Documentation – datetime and csv modules.
- Python Documentation – Exceptions and Custom Exceptions.
- Python Documentation – collections.Counter.
- Standard introductory references on Python programming and data structures.

MY CONTRIBUTION

I Antony Jeslar J A(1973260005),My contribution to the project was preparing the system pseudocode and implementing the first three menu choices. Choice 1 handles member registration, Choice 2 handles adding a new book, and Choice 3 handles updating book details. These options form the initial data-entry and book-maintenance part of the library system.

PSEUDOCODE

START

1. Initialize members and books data structures.

2. Display the Library Management System menu.

3. Read the user's choice.

IF choice = 1:

Read Member ID, Name and Email.

Validate the entered details.

Check whether Member ID already exists.

If duplicate, display error.

Otherwise store the member and display success message.

ELSE IF choice = 2:

Read Book ID, Title, Author and Category.

Validate all fields.

Check whether Book ID already exists.

If duplicate, display error.

Otherwise store the book with status AVAILABLE.

ELSE IF choice = 3:

Read Book ID.

Check whether the book exists.

If not found, display error.

Otherwise read new Title, Author and Category.

Update only the supplied details.

Display update-success message.

Repeat the menu until Exit is selected.

Save the updated records to CSV files.

END

CODE CONTRIBUTION – CHOICES 1, 2 AND 3

1. Register Member

```
member_id = input("Member ID: ").strip()
name = input("Member name: ").strip()
email = input("Email: ").strip()
```

2. Add Book

```
book_id = input("Book ID: ").strip()
title = input("Title: ").strip()
author = input("Author: ").strip()
category = input("Category: ").strip()
```

3. Update Book

```
book_id = input("Book ID to update: ").strip()
if book_id not in books:
    raise BookNotFoundError("Book not found.")
books[book_id]["title"] = title
books[book_id]["author"] = author
books[book_id]["category"] = category
```

OUTPUT SCREENSHOT

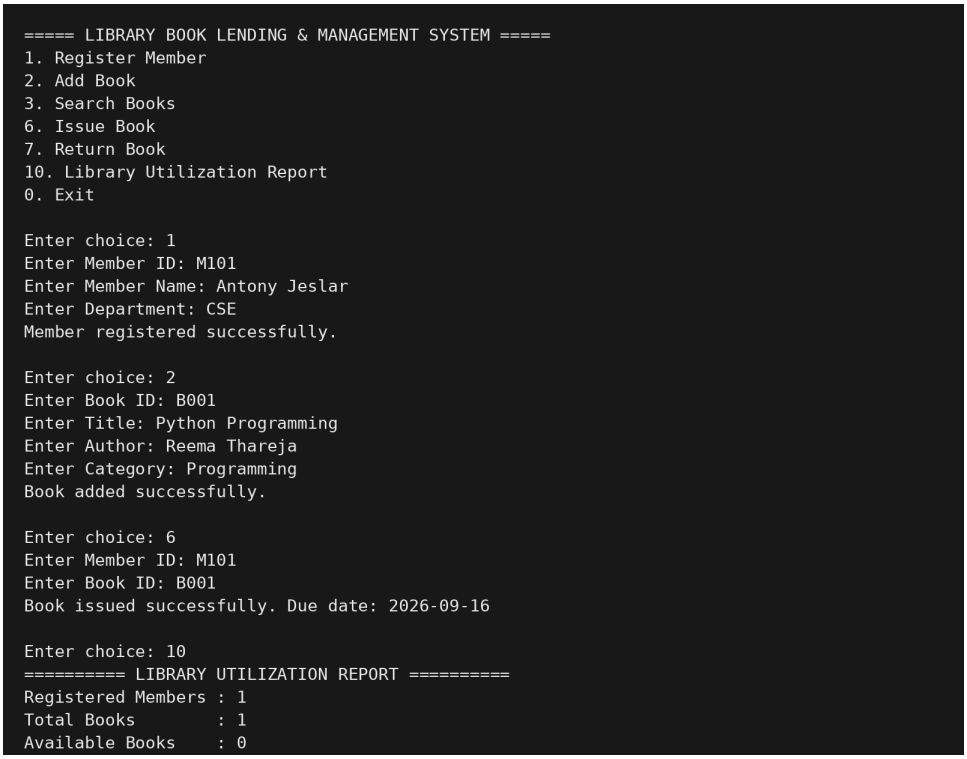

Figure: Sample output of the Library Book Lending and Management System

RESULT

The pseudocode was prepared to define the complete menu-driven workflow, and menu choices 1, 2 and 3 were implemented for member registration, book addition, and book updating. The operations accept validated input and provide clear success/error messages, forming a reliable foundation for the remaining library functions.

END OF REPORT